

SAÉ 2.01-SAÉ 2.02

1.04 : Rapport de test automatisé JUnit

Groupe A1

Brian LOUBASSOU

Fatou Bintou SALL

Hadja Haidara

Arthur alimba

Année universitaire 2025/2026

Introduction:

Dans le cadre de la SAÉ 2.01, nous avons réalisé des tests automatisés avec JUnit afin de vérifier le bon fonctionnement des principales méthodes de notre simulation de robots mineurs.

L'objectif des tests est de s'assurer que les robots respectent les règles du sujet : se déplacer uniquement sur des cases autorisées, récolter uniquement dans les mines de leur spécialité et respecter les capacités de stockage et d'extraction.

Nous avons choisi principalement la méthode des classes d'équivalence, complétée par quelques tests aux limites.

❖ Méthodes testées

Les deux méthodes principales testées sont :

Robot.recolter(Mine mine)

Monde.deplacerRobot(Robot robot, Direction direction)

❖ Critères de test choisis

Pour tester notre application, nous avons principalement utilisé le critère des **classes d'équivalence**.

Cette méthode consiste à regrouper les données d'entrée en différentes classes, valides ou invalides, produisant un comportement similaire. Cela permet de réduire le nombre de tests à réaliser tout en conservant une bonne couverture des cas possibles.

Nous avons également utilisé des **tests aux limites** afin de vérifier le comportement du programme sur les valeurs situées aux bornes des domaines autorisés, par exemple les limites de la grille du monde ou les capacités minimales et maximales des robots.

Ces deux approches permettent de tester efficacement les principales fonctionnalités de l'application sans avoir à vérifier toutes les combinaisons possibles de données d'entrée.

❖ **Tableau des jeux de tests**

Méthode Robot .récolter(Mine mine)

N°	Critère utilisé	Donnée d'entrée	Résultat attendu
1	Classe d'équivalence valide	Robot OR, mine OR avec 80 minerais, capacité extraction = 2, robot vide	Récolte acceptée. Stock robot = 2. Mine = 78
2	Classe d'équivalence invalide	Robot OR, mine NICKEL avec 80 minerais	Récolte refusée. Stock robot = 0. Mine = 80
3	Classe d'équivalence invalide	Robot OR, mine OR vide	Récolte refusée. Stock robot = 0. Mine = 0
4	Test limite	Robot capacité 5, stock déjà à 3, extraction = 3	Robot prend seulement 2. Stock final = 5
5	Test limite	Mine OR contient 1 minerais, robot extraction = 3	Robot prend seulement 1. Mine = 0

Méthode Monde.deplacerRobot(Robot robot, Direction direction)

N°	Critère utilisé	Donnée d'entrée	Résultat attendu
1	Classe d'équivalence valide	Robot en (1,1), déplacement EST vers terrain libre	Déplacement accepté. Robot en (1,2)
2	Classe d'équivalence invalide	Robot en (1,1), eau en (1,2), déplacement EST	Déplacement refusé. Robot reste en (1,1)
3	Classe d'équivalence invalide	Robot en (1,1), autre robot en (1,2), déplacement EST	Déplacement refusé. Robot reste en (1,1)
4	Test limite	Robot en (0,0), déplacement NORD	Déplacement refusé. Robot reste en (0,0)
5	Test limite	Robot en (9,9), déplacement SUD	Déplacement refusé. Robot reste en (9,9)

❖ Analyse des tests réalisés

Les tests réalisés respectent les principes des classes d'équivalence et des tests aux limites.

Pour les classes d'équivalence, nous avons identifié des cas valides (déplacement sur terrain libre, récolte correcte) ainsi que des cas invalides (déplacement vers l'eau, vers un robot, ou mine de mauvais type).

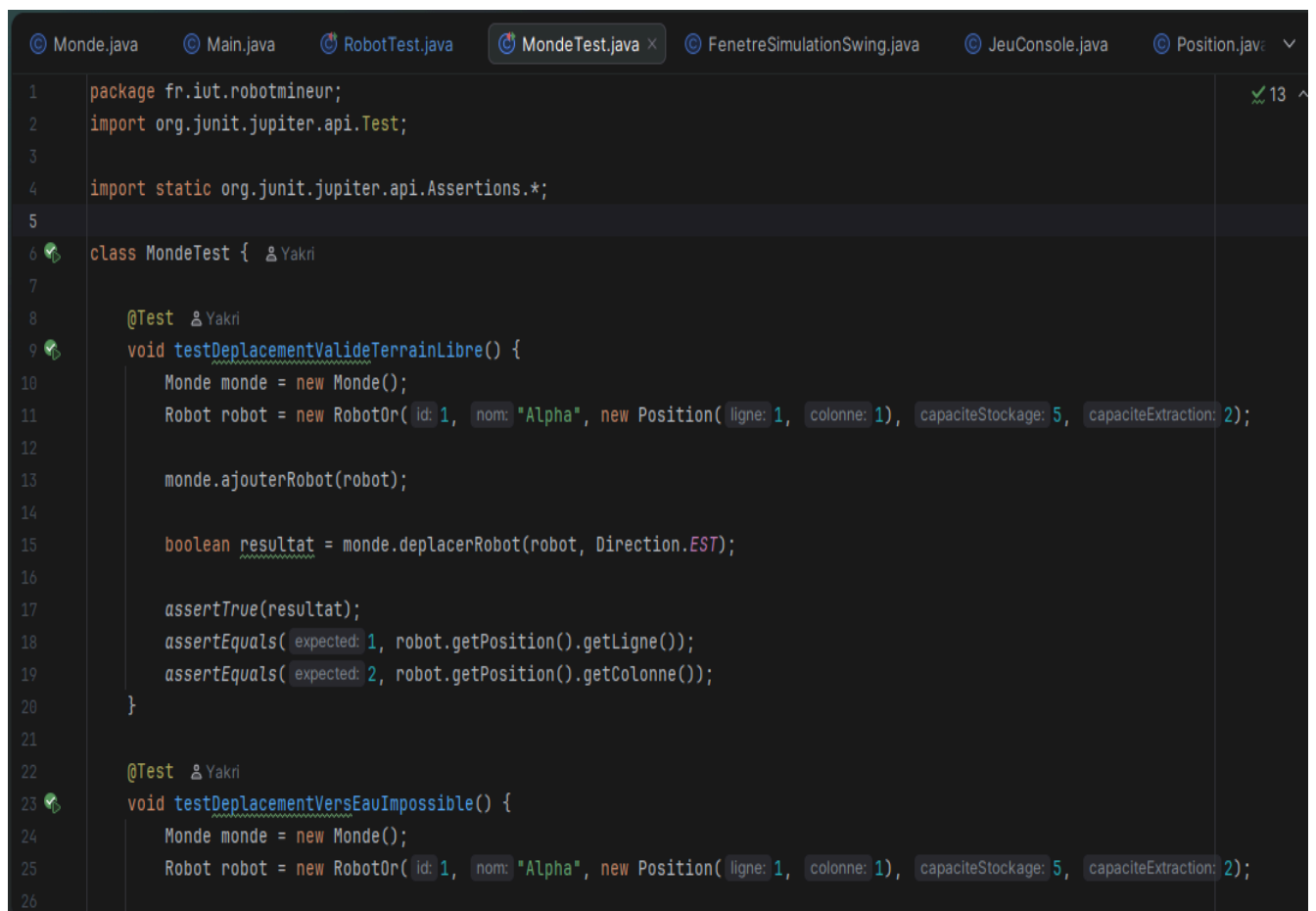
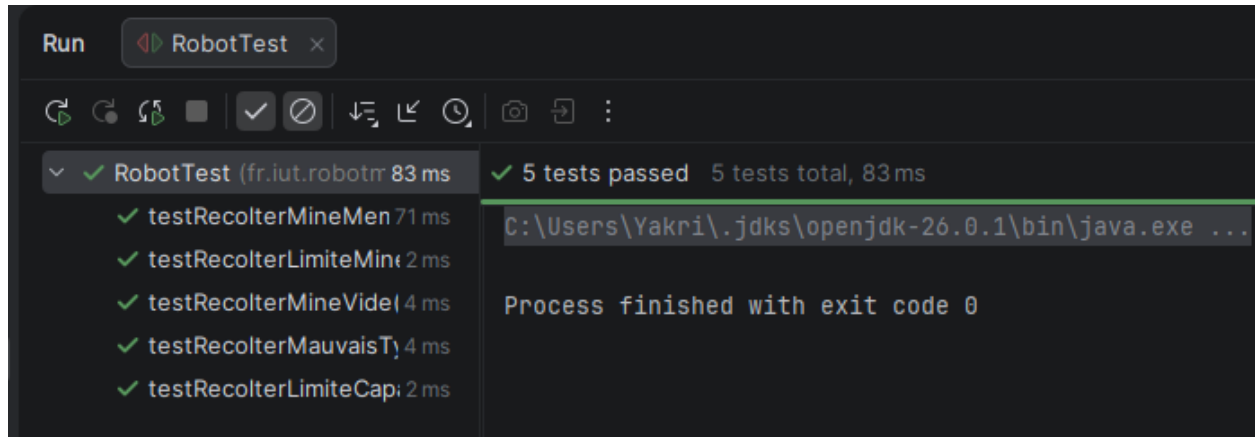
Concernant les tests aux limites, nous avons vérifié les déplacements en bordure de la grille (lignes 0 et 9), ainsi que les capacités maximales des robots et des mines.

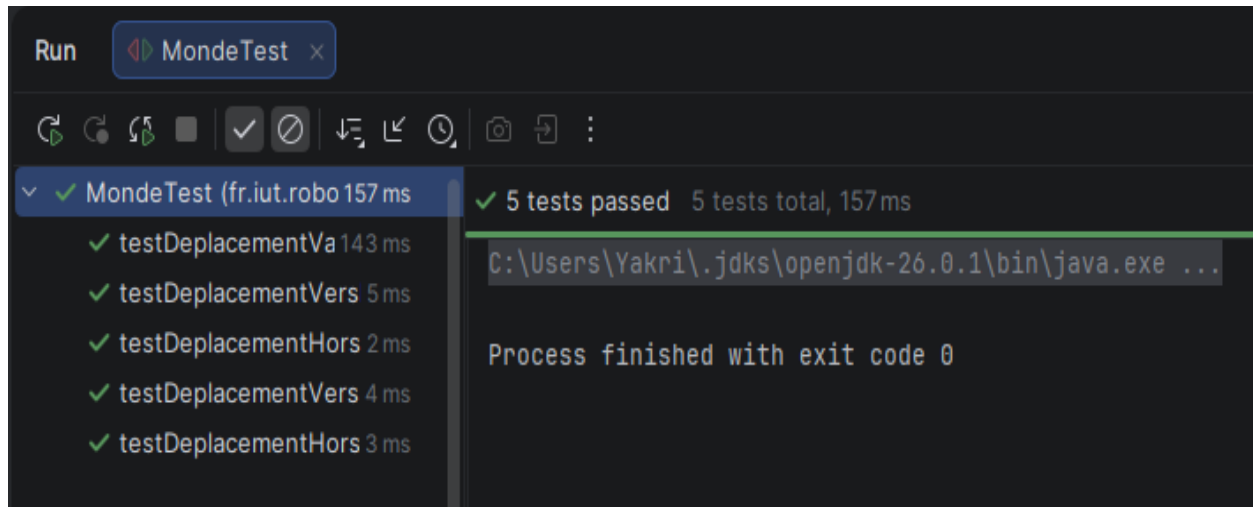
Ces tests permettent de couvrir les principaux comportements du système.

❖ Implementation JUnit et captures des résultats des tests

```
Monde.java Main.java RobotTest.java MondeTest.java FenetreSimulationSwing.java JeuConsole.java Position.java
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 class RobotTest {
9
10     @Test
11     void testRecolterMineMemeType() {
12         Robot robot = new RobotOr(id: 1, nom: "Alpha", new Position(ligne: 0, colonne: 0), capaciteStockage: 5, capaciteExtraction: 2);
13         Mine mine = new Mine(id: 1, new Position(ligne: 0, colonne: 0), TypeMinerai.OR, quantiteActuelle: 80, quantiteInitiale: 80);
14
15         boolean resultat = robot.recolter(mine);
16
17         assertTrue(resultat);
18         assertEquals(expected: 2, robot.getStockActuel());
19         assertEquals(expected: 78, mine.getQuantiteActuelle());
20     }
21
22     @Test
23     void testRecolterMauvaisType() {
24         Robot robot = new RobotOr(id: 1, nom: "Alpha", new Position(ligne: 0, colonne: 0), capaciteStockage: 5, capaciteExtraction: 2);
25         Mine mine = new Mine(id: 1, new Position(ligne: 0, colonne: 0), TypeMinerai.NICKEL, quantiteActuelle: 80, quantiteInitiale: 80);
26
27         boolean resultat = robot.recolter(mine);
28
29         assertFalse(resultat);
30     }
31 }
```

lut > robotmineur > RobotTest 7:1 CRLF UTF-8 4 spaces





Conclusion

Les tests réalisés dans le cadre de ce projet ont permis de valider le bon fonctionnement des principales fonctionnalités de l'application de simulation de robots mineurs.

En utilisant les méthodes des classes d'équivalence et des tests aux limites, nous avons pu couvrir à la fois les cas normaux d'utilisation ainsi que les situations extrêmes ou invalides. Les tests ont notamment porté sur les déplacements des robots, la récolte des minerais et les interactions avec l'environnement (eau, mines, autres robots).

L'implémentation des tests automatisés avec JUnit a permis de vérifier efficacement ces comportements. Les résultats obtenus montrent que toutes les méthodes testées fonctionnent conformément aux attentes, comme en témoigne l'exécution réussie de l'ensemble des tests.

Ainsi, cette démarche de test contribue à garantir la fiabilité et la robustesse de l'application développée.